AGROGUARDIAN

Project Overview

An AI-assisted agricultural monitoring and irrigation concept designed around a simple loop: **Sense** → **Understand** → **Decide** → **Act** → **Observe again.**

Project focus	Physical AI for agricultural environments
Initial approach	Simulation-first, with a path toward sensors and hardware
AI direction	Nebius Token Factory inference with a suitable NVIDIA open model, subject to availability and testing
Current status	Concept and development plan; hardware and model integration are not yet confirmed

Prepared as a working overview to guide implementation, testing, and eventual hackathon presentation.

1. Project Summary

AgroGuardian is a proposed AI-assisted system for monitoring agricultural conditions and helping manage irrigation. It combines environmental readings and, where available, camera observations with a decision process that can recommend safe actions. The first version will use simulated sensor data so the software can be developed and tested before physical hardware is introduced.

The goal is to demonstrate a complete, observable loop: the system receives measurements, interprets the current situation, chooses from a limited set of actions, applies safety checks, performs a simulated action, and records what happened.

2. Problem and Proposed Value

Agricultural environments can change throughout the day. Soil moisture, temperature, humidity, light, and obstacles may affect irrigation decisions or the safe movement of a mobile platform. A monitoring system can bring these signals together and provide a clearer operational picture.

Proposed value

- Combine several sensor readings in one place.
- Identify conditions that may require attention, such as low soil moisture or excessive temperature.
- Recommend bounded actions, such as a short irrigation cycle or an alert to the operator.
- Keep a record of observations, decisions, safety checks, and actions for review.

3. Core Operating Loop

1. SENSE Read simulated or real sensors.	**2. UNDERSTAND** Summarize the current conditions.	**3. DECIDE** Choose an allowed action.	**4. ACT** Run the safety-checked action.	**5. OBSERVE** Check and log the result.

Illustrative inputs: temperature, humidity, soil moisture, light level, camera observations, and distance to an obstacle. Exact sensors and thresholds will be decided during implementation and testing.

4. Proposed Architecture

Sensor layer	Initially simulated sensor readings; later, physical sensors or a robotics simulator.
Agent / orchestration	Python code gathers readings, prepares context, requests a model recommendation, and logs the result.
Model inference	Nebius Token Factory API calling a suitable NVIDIA open model, such as a compatible Nemotron model, after checking current availability and API details.
Safety controller	Deterministic rules validate every proposed action against thresholds, actuator availability, and time/duration limits.
Action layer	Initially simulated actions; later, controlled interfaces to a pump, alert system, or mobile platform.

Monitoring / logs	Readable terminal output first, with a possible dashboard added later.

5. Safety and Reliability

The language model should not directly control motors or irrigation hardware. It should only propose an action from a defined list. A separate deterministic controller must approve, modify, or reject that proposal before any action is executed.

- Set explicit minimum and maximum limits for irrigation duration and other actuator commands.
- Reject unknown actions and malformed model responses.
- Check sensor validity and actuator availability before execution.
- Use safe defaults when readings or model responses are missing.
- Log the sensor values, proposed action, safety decision, executed action, and result.
- Keep all hardware disconnected or in simulation mode until the safety checks are tested.

6. Initial Software Structure

A small Python repository keeps the first milestone easy to run and understand. The structure can expand when a dashboard, robotics simulator, or hardware interface is added.

```
agroguardian/
■■■ backend/
■ ■■■ agent/
■ ■ ■■■ agent.py
■ ■■■ tools/
■ ■ ■■■ robot_tools.py
■ ■■■ safety/
■ ■ ■■■ controller.py
■ ■■■ sensors/
■ ■■■ simulated_sensors.py
■■■ tests/
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
■■■ run.py
```

The actual repository may differ as implementation proceeds. Keep API keys in a local .env file, exclude that file from Git, and commit only a safe .env.example containing placeholder values.

7. Development Roadmap

Stage	Deliverable	Completion check
1. Local prototype	Simulated sensors and deterministic safety rules.	The program runs and prints readings plus a safe action.
2. Model integration	Call a compatible NVIDIA model through Nebius Token Factory.	The runtime makes a real API call and handles failures.
3. Action tools	Define a small allowlist of possible actions.	Unsafe, unknown, or out-of-range actions are rejected.
4. Tests and logs	Test edge cases and record decisions.	Tests cover invalid readings, timeouts, and safety limits.
5. Visual demo	Add a dashboard or simulator view.	A reviewer can see inputs, decisions, and actions.
6. Optional hardware	Connect sensors or a rover/pump only when available.	Hardware is tested with conservative limits and a stop mechanism.

8. Hackathon Alignment Checklist

The hackathon brief shared during planning calls for a working application using Nebius infrastructure and at least one NVIDIA open-source model. The following is a planning checklist, not a claim that these items have already been completed.

- Demonstrate a real runtime API call to Nebius Token Factory or a deployment on Nebius AI Cloud.
- Use and clearly identify at least one qualifying NVIDIA open-source model.
- Publish a code repository with setup instructions, assets as appropriate, a README, and an open-source license.
- Provide a working demo or hosted app/test build.
- Record a public demo video under three minutes that shows the system working. If no physical hardware is used, show the key application modules in action.
- Explain the problem, the solution, technical choices, and the expected impact.

9. Risks and Open Decisions

- Model selection, availability, pricing, and API format must be verified in the Nebius account before integration.
- Physical hardware, sensors, pump, camera, and robotics platform have not been confirmed.
- Simulation can demonstrate software logic but does not prove real-world hardware reliability.
- Agricultural thresholds depend on crop, soil, climate, sensor calibration, and operating conditions; they should not be hard-coded as universal agronomic advice.
- A network or model failure must not cause an unsafe actuator command.

10. Definition of the First Working Demo

The first useful milestone is a command-line application that generates sample environmental readings, requests a constrained recommendation from the configured model when available, passes the recommendation through

the local safety controller, executes only a permitted simulated action, and prints a clear log of the complete cycle. If the model call fails, the program should fail safely and still report the issue.

This overview describes the planned design. Model integration, hardware support, deployment, and hackathon submission requirements remain implementation tasks until demonstrated and verified.